Learning AFG & DSO using Lissajous Figures

Jnanesh Sathisha Karmar - EE25BTECH11029
Vivek Kumar - EE25BTECH11062

January 19, 2026

0.1 Explain the basic functions of an Arbitrary Function Generator (AFG). What parameters can be controlled?

An arbitrary function generator can be used to generate any form of input signal. It can be used to control the wave shape. Different wave shapes come included such as square, ramp, sine, pulse and noise, and many other wave shapes. There is also an option to choose an arbitrary waveform, where we can generate any waveform of our choice.

For each wave we can adjust the wave parameters like frequency, intital phase, amplitude, offset.

- **Frequency**: It is the number of wave cycles that passes a fixed point per second. Along with frequency, there is an option to change the time period (inverse of frequency).
- **Initial Phase**: We can adjust the initial phase of the signal. This is useful for creating a phase difference between 2 signals.
- **Amplitude**: We can also adjust the amplitude of the waveform in terms of Vpp(Voltage peak-peak). In the model available in the lab, the maximum amplitude that we can set is 10 Vpp.
- **Offset**: We can also shift the waveform vertically using this option.

The model available in lab can be used to produce atmost 2 signals, where each one of them can be independently controlled. There is also a numpad and a dial to manually adjust each parameter.

0.2 What are V/div and Time/div in a Digital Storage Oscilloscope (DSO)? Why are they important?

They represent the scales of the time-domain graph used to visualize the waveform. V/div refers to Voltage/division, which is the y-axis scale. It helps us analyze the

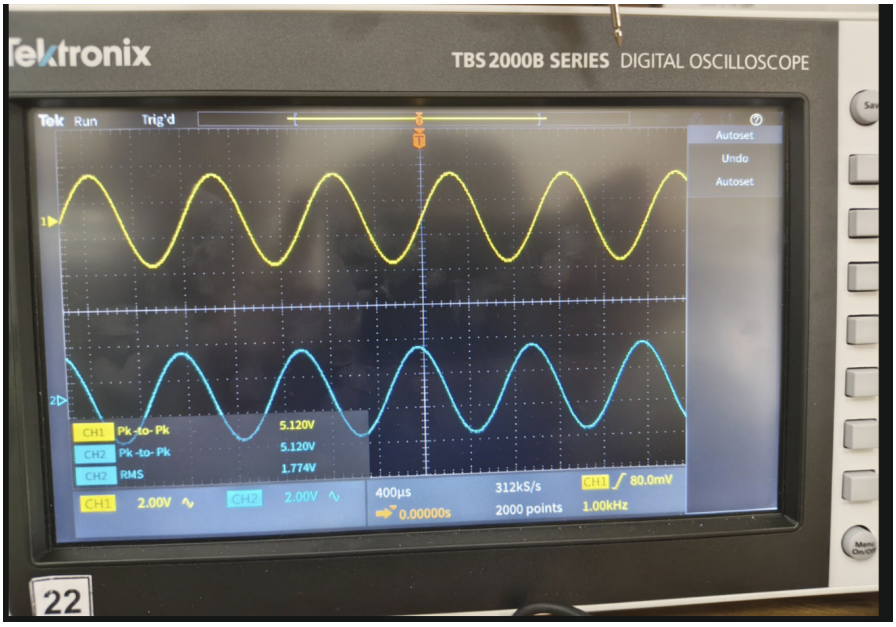


Figure 1: Example figure for V/div and Time/div

amplitude of the waveform. Time/div is the x-axis scale using which, we can infer the frequency of the waveform.
In Figure 1, the V/div is 2V and we can see that the waveform approximately occupies 2.5 divisions in the y-direction. It corresponds to the 5V peak-to-peak amplitude. Similarly, each wave cycle occupies 2.5 divisions in the x-direction, and Time/div is $400s$. It corresponds to the time period of $1ms$.

0.3 Define phase difference between two sinusoidal signals. How can it be measured using time-domain analysis?

Phase difference between the two sinusoidal signals can be defined as the difference in the initial phase angle of the two waves (on a circle).

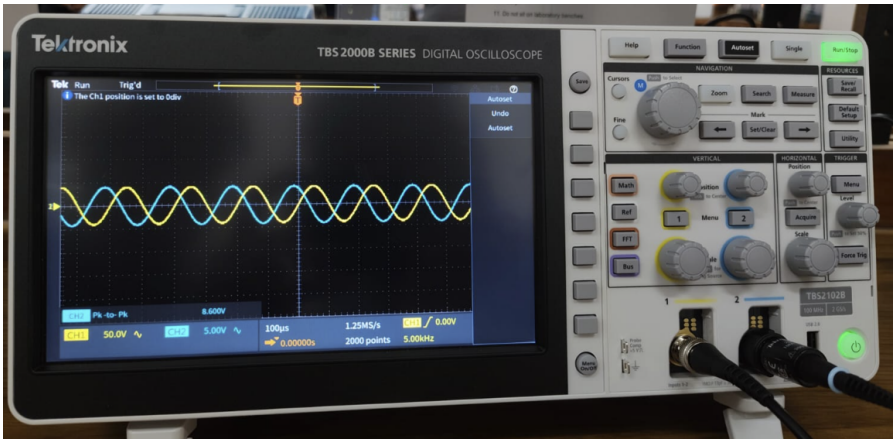
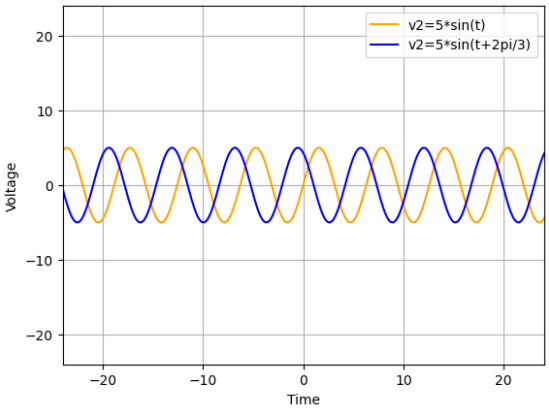


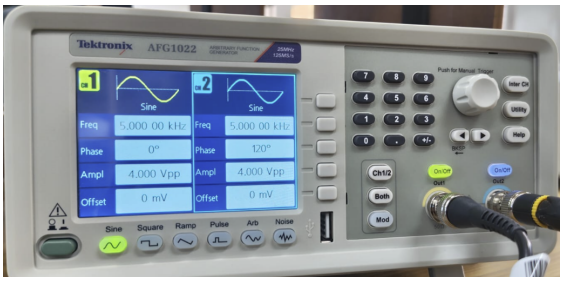
Figure 2: Finding phase difference using DSO

We can find the phase difference of two signals with the same frequency by finding the peak-to-peak displacement in terms of phase. In Figure 2, the time period is 10 div, and the time between two closest peaks is around 3.33 div.
Hence in this case, $\Delta\phi = \frac{3.33}{10} \times 2\pi = \frac{2\pi}{3}$. We can verify the above result by looking at the AFG used to generate these signals.

A more general formula for finding phase difference can be expressed as

$$\Delta\phi = \frac{\text{Time between two closest peaks}}{\text{Time Period}} \times 2\pi \tag{0.1}$$





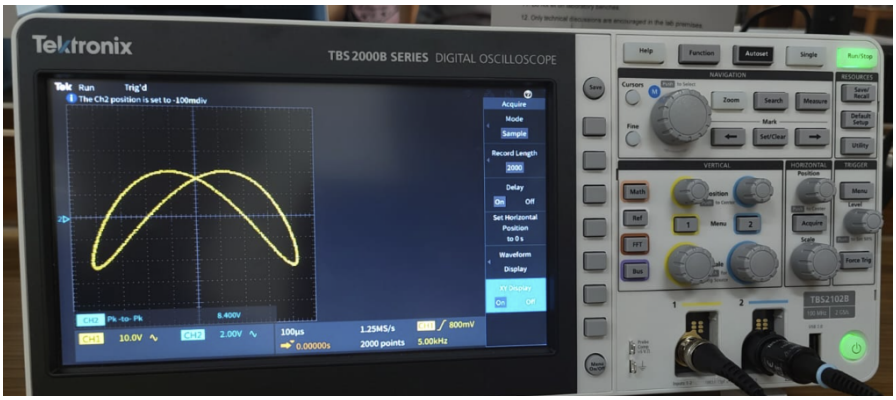
The python code to plot the above signal is given below.

```
import numpy as np
import matplotlib.pyplot as plt
```

```
t = np.linspace(-20*np.pi, 20*np.pi, 1000)
x = 5*np.sin(t)
y = 5*np.sin(t + 2*np.pi/3)

plt.plot(t, x, label = 'v2=5*sin(t)', color='orange')
plt.plot(t, y, label = 'v2=5*sin(t+2pi/3)', color='blue')
plt.xlim(-24, 24)
plt.ylim(-24, 24)
plt.xlabel('Time')
plt.ylabel('Voltage')
plt.grid()
plt.legend()
plt.show()
```

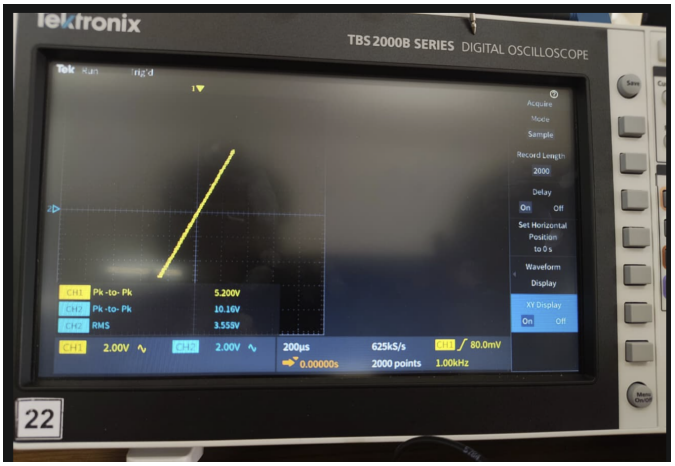
0.4 What is XY mode in a DSO? How is it different from time-domain mode?

XY mode is a useful tool which can be used to compare both the signals. Instead of the time-domain mode which shows the two waveforms as the function of time, the XY mode has one signal as a function of another. The input signal from channel-1 is the x-axis and the input signal from channel-2 is the y-axis.



The resulting shapes in the XY mode are called lissajous figures. Which can be useful to measure phase-difference and compare frequencies of two signals. The time-domain mode can also be viewed as applying a sawtooth waveform in the X direction.

0.5 Draw and explain Lissajous figures for (a) 0° phase difference and (b) 90° phase difference



(a) Straight line for 0° phase difference

The lissajous figures for 0° phase difference is a bounded straight line, as we can see in Figure 4a. The slope is the ratio of the amplitudes.

$$x = A_1 \sin(\omega t) \tag{0.2}$$

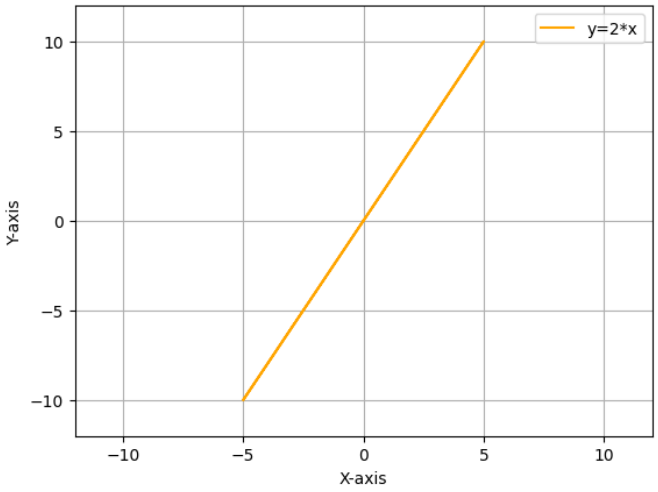
$$y = A_2 \sin(\omega t) \tag{0.3}$$

$$\frac{x}{y} = \frac{A_1}{A_2} \tag{0.4}$$

$$y = \frac{A_2}{A_1} x \tag{0.5}$$

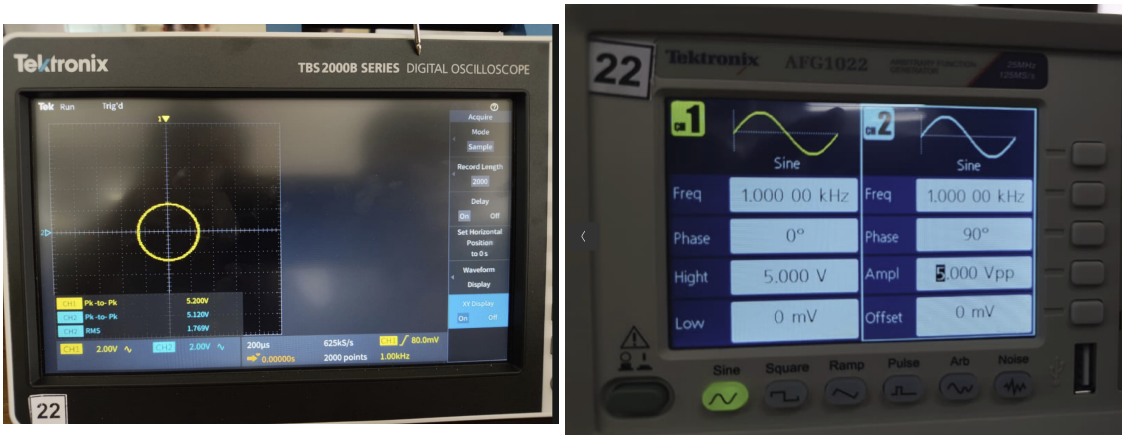
$$m = \frac{A_2}{A_1} \tag{0.6}$$

The python plot for the above is given below.



The python code used to generate the above plot is given below:

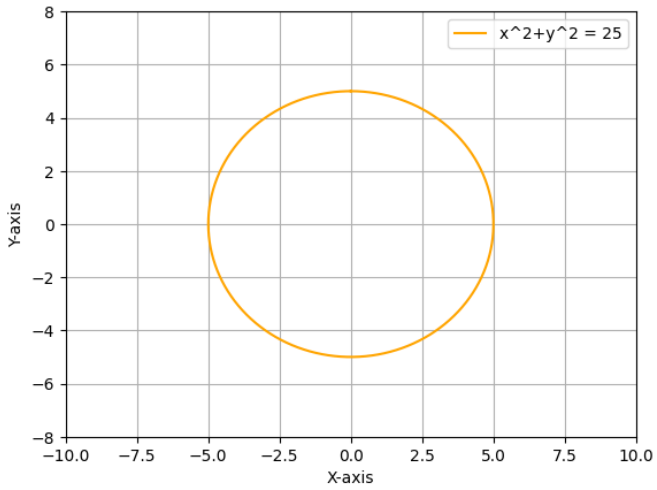
```
import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(0, 2*np.pi, 1000)
x = 5*np.sin(t)
y = 10*np.sin(t)

plt.plot(x, y, label = 'y=2*x', color='orange')
plt.xlim(-12, 12)
plt.ylim(-12, 12)
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.grid()
plt.legend()
plt.show()
```



(a) Lissajous figure for 90° phase difference

The lissajous figures for 90° phase difference is a circle centered at origin if the amplitudes are equal, as we can see in figure5a.

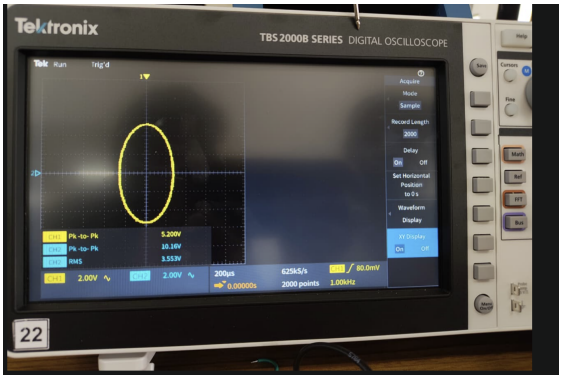
The plot for the above is given below.



The python code for the above plot is given below.

```
import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(0, 2*np.pi, 1000)
x = 5*np.sin(t)
y = 5*np.cos(t)

plt.plot(x, y, label = 'x^2+y^2=25', color='orange')
plt.xlim(-10, 10)
plt.ylim(-8, 8)
```

```
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.grid()
plt.legend()
plt.show()
```



The lissajous figures for 90° phase difference is an ellipse with axes along coordinate axes if the amplitudes are different, as we can see in the figure above. The eccentricity of the ellipse depends on the ratio of the amplitudes.

$$x = A_1 \sin(\omega t) \tag{0.7}$$

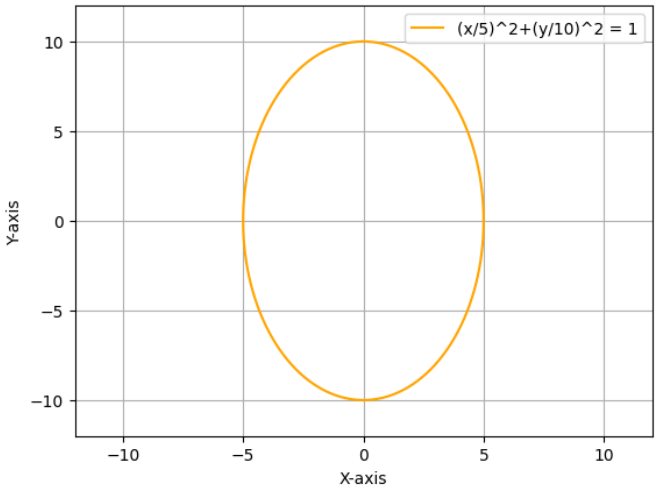
$$y = A_2 \sin(\omega t + \frac{\pi}{2}) \tag{0.8}$$

$$= A_2 \cos(\omega t) \tag{0.9}$$

$$(\frac{x}{A_1})^2 + (\frac{y}{A_2})^2 = 1 \tag{0.10}$$

$$e = \sqrt{1 - (\frac{A_1}{A_2})^2}, \quad \text{if } A_1 < A_2 \tag{0.11}$$

The plot for the above is given below.

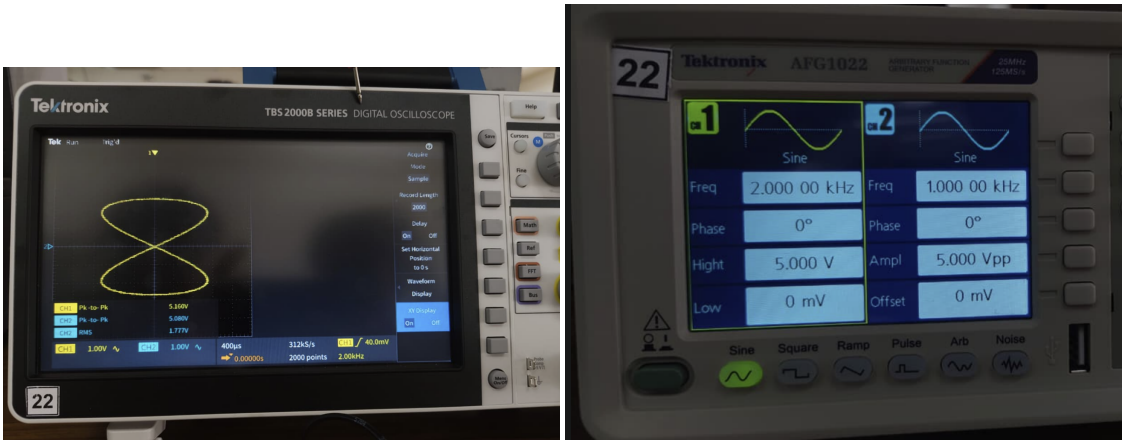


The python code for the above plot is given below.

```
import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(0, 2*np.pi, 1000)
x = 5*np.sin(t)
y = 10*np.cos(t)

plt.plot(x, y, label = '(x/5)^2+(y/10)^2=1', color='orange')
plt.xlim(-12, 12)
plt.ylim(-12, 12)
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.grid()
plt.legend()
plt.show()
```

0.6 Explain how Lissajous figures are used to determine frequency ratio.

Assuming that the two signals are sinusoidal and have the same phase, there is a way to find the frequency ratio of two signals using lissajous figures. There are many cases:
Case 1: Simple loops

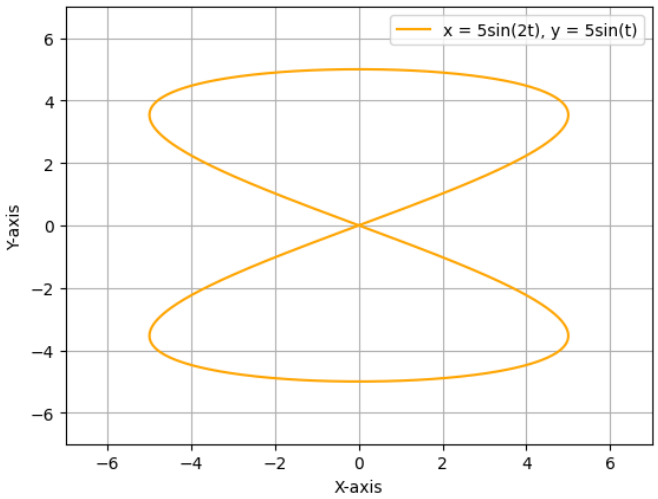


If the lissajous figure comprises only of simple loops like the figure above, then we can say that the frequency ratio is an even number.

The frequency ratio would simply be the number of simple loops visible. If we define the frequency of the first signal to be $\mathbf{f_1}$ and the frequency of the second signal to be $\mathbf{f_2}$ and, let $\mathbf{n_y}$ be the number of simple loops along the y-direction and $\mathbf{n_x}$ be the number of simple loops along the x-direction, then.

$$\frac{\mathbf{f_1}}{\mathbf{f_2}} = \begin{cases} n_y, & \text{if the loops are along the y-direction} \\ \frac{1}{n_x}, & \text{if the loops are along the x-direction} \end{cases} \tag{0.12}$$

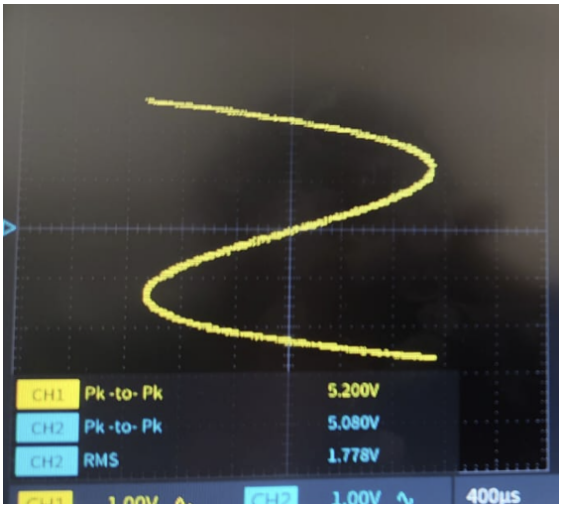
The python plot for the above is given below.

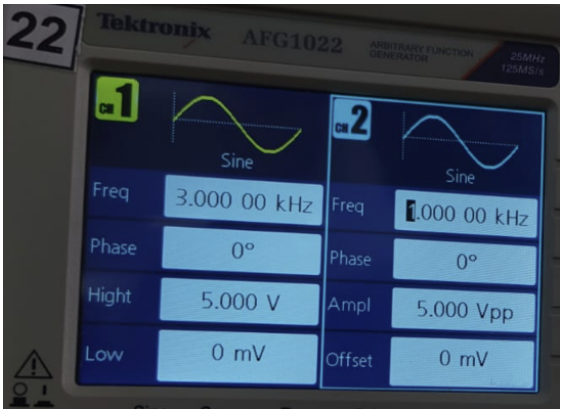


The python code used to generate the above plot is given below:

```
import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(0, 2*np.pi, 1000)
x = 5*np.sin(2*t)
y = 5*np.sin(t)

plt.plot(x, y, label = 'x=5sin(2t),y=5sin(t)', color='orange
    ')
plt.xlim(-7, 7)
plt.ylim(-7, 7)
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.grid()
plt.legend()
plt.show()
```

Case 2: Simple open curve

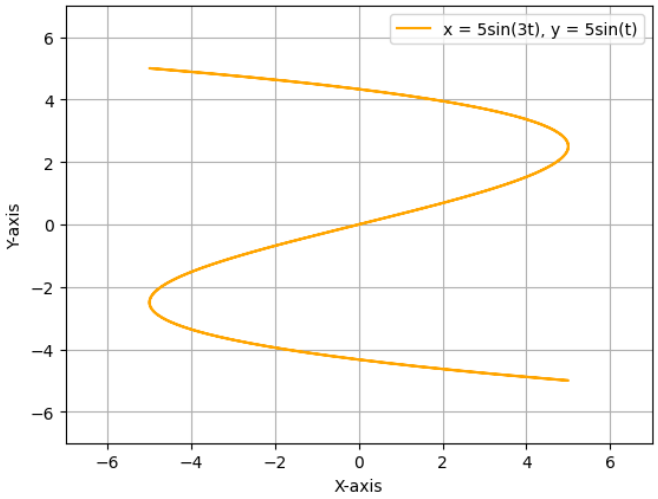




If the lissajous figure is a simple open curve, and the curve does not intersect itself at any point, then the number of times it crosses the co-ordinate axis is the frequency ratio.
$\mathbf{f_1}$ is the frequency of the first signal, and $\mathbf{f_2}$ is the frequency of the second signal, then

$$\frac{\mathbf{f_1}}{\mathbf{f_2}} = \frac{\text{Number of times curve crosses y-axis}}{\text{Number of times curve crosses x-axis}} \tag{0.13}$$

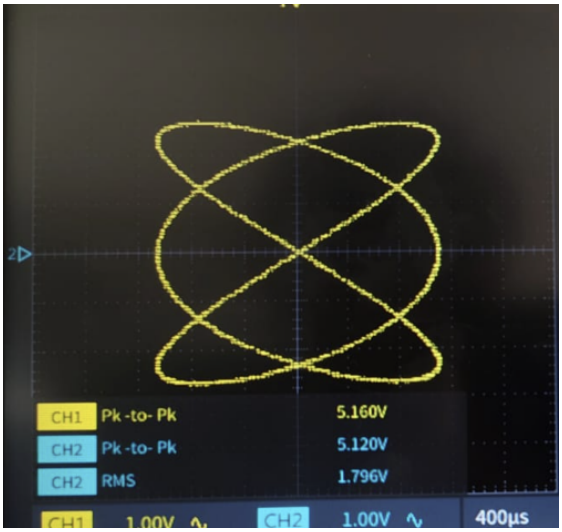
The python plot for the above is given below.

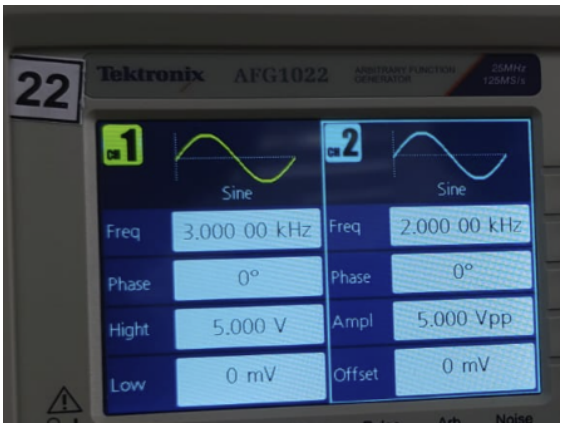


The python code used to generate the above plot is given below:

```
import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(0, 2*np.pi, 1000)
x = 5*np.sin(3*t)
y = 5*np.sin(t)

plt.plot(x, y, label = 'x=5sin(3t),y=5sin(t)', color='orange
    ')
plt.xlim(-7, 7)
plt.ylim(-7, 7)
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.grid()
plt.legend()
plt.show()
```

Case 3: General closed curve with loops



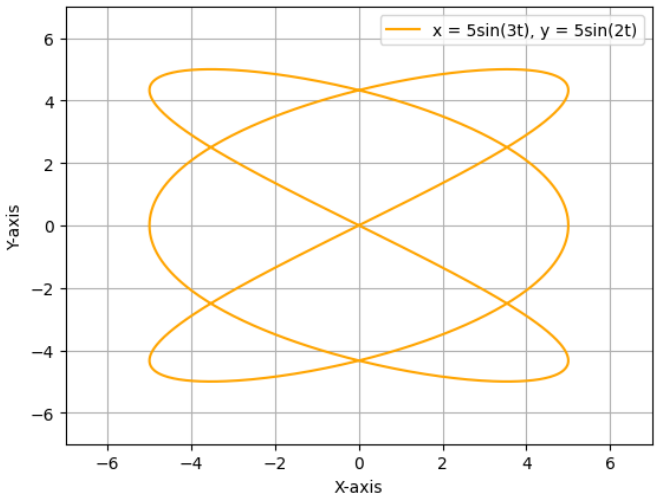


This can also be applied to the simple loops case. In this case, find the ratio of the number of loops in a direction parallel to the corresponding axis. We can also find it by imagining the rectangular edges around the lissajous figure. The ratio of the number of times the vertical edge touches the curve to the number of times the horizontal edge touches the curve is the required ratio.

If we define the frequency of the first signal to be $\mathbf{f_1}$ and the frequency of the second signal to be $\mathbf{f_2}$, then

$$\frac{\mathbf{f_1}}{\mathbf{f_2}} = \frac{\text{Number of times curve touches tangent parallel to y-axis}}{\text{Number of times curve touches tangent parallel to x-axis}} \tag{0.14}$$

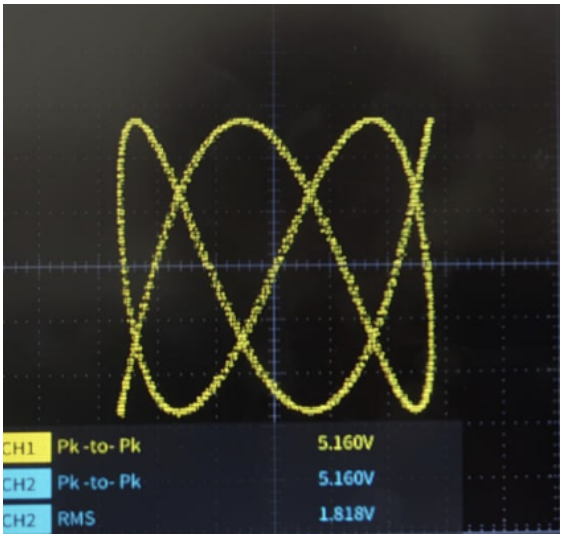
The python plot for the above is given below.

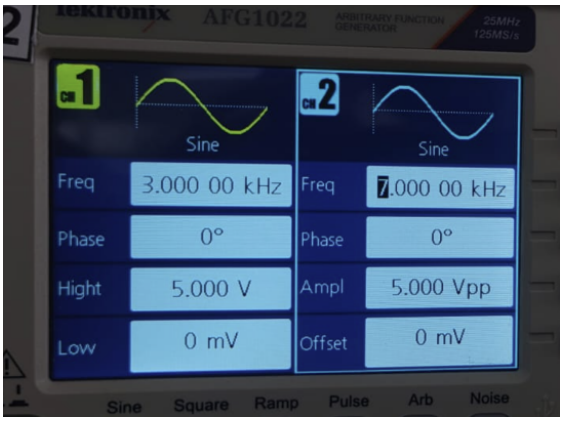


The python code used to generate the above plot is given below:

```
import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(0, 2*np.pi, 1000)
x = 5*np.sin(3*t)
y = 5*np.sin(2*t)

plt.plot(x, y, label = 'x=5sin(3t),y=5sin(2t)', color = '
    orange')
plt.xlim(-7, 7)
plt.ylim(-7, 7)
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.grid()
plt.legend()
plt.show()
```

Case 4: General open curve with loops



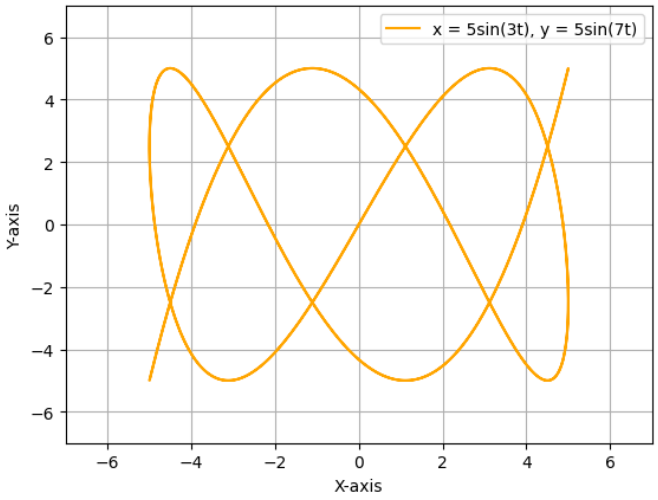


This can also be applied to the simple open curve case. In this case, The ratio of the number of times the curve crosses the y-axis to the number of times the curve crosses the x-axis is the required ratio.

If we define frequency of the first signal to be $\mathbf{f_1}$, and the frequency of the second signal to be $\mathbf{f_2}$.

$$\frac{\mathbf{f_1}}{\mathbf{f_2}} = \frac{\text{Number of times curve crosses y-axis}}{\text{Number of times curve crosses x-axis}} \tag{0.15}$$

The python plot for the above is given below.



The python code used to generate the above plot is given below:

```
import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(0, 2*np.pi, 1000)
x = 5*np.sin(3*t)
y = 5*np.sin(7*t)

plt.plot(x, y, label = 'x=5sin(3t),y=5sin(7t)', color='
    orange')
plt.xlim(-7, 7)
plt.ylim(-7, 7)
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.grid()
plt.legend()
plt.show()
```

0.7 Mention two practical applications of lissajous figures

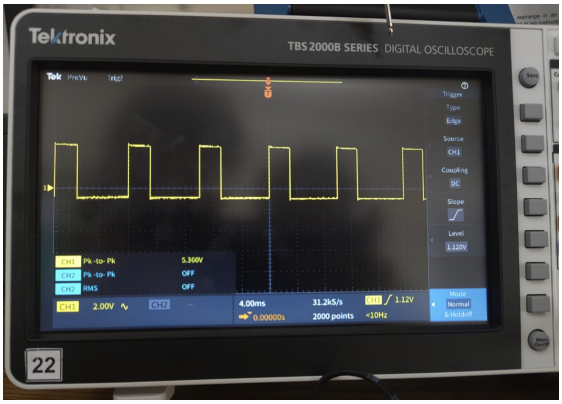
- It can be used to find the frequency ratio of two sinusoidal signals, if all we know about them is that they have the same phase (i.e, 0 phase difference).

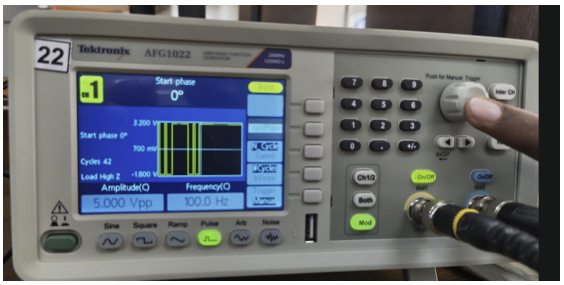
- It can be used to find the amplitude ratio of two sinusoidal signals, if all we know about them is that they have the same frequency and phase.
- It can be used to visualize SHM in different directions.

0.8 Explain different triggering modes available in a DSO (edge, level, pulse, etc.) How are triggers used to capture single-shots or non-repetitive events?

The trigger in a DSO is a tool to capture any particular event, such as

- A sudden rise (edge)
- A pulse of a particular width
- A runt (a smaller/different pulse among a set of pulses).



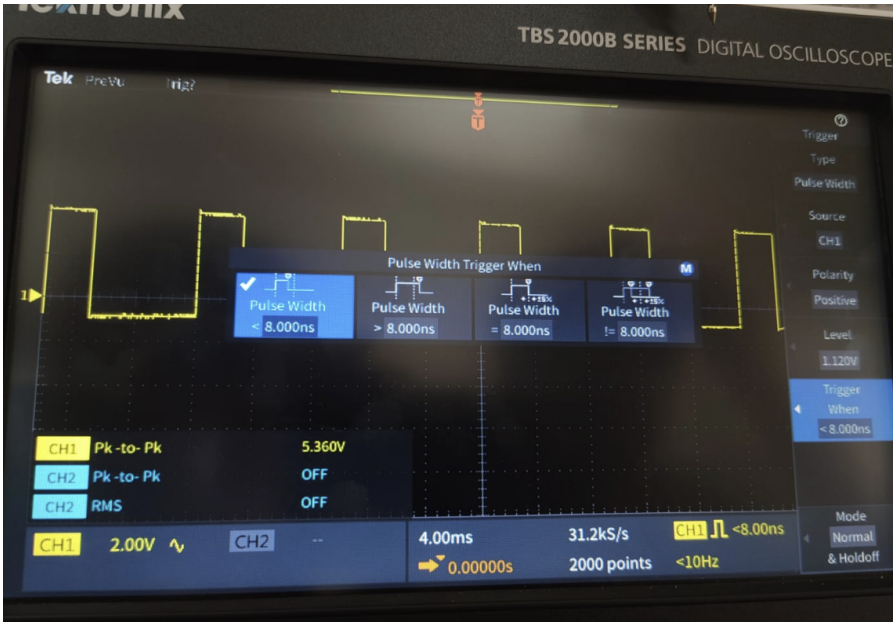


We use an AFG in burst mode to generate a set of pulses for the DSO to detect. The DSO, will then create a snapshot of the signal, the instant it detects the edge.

The different edge trigger options are shown in the figure below.

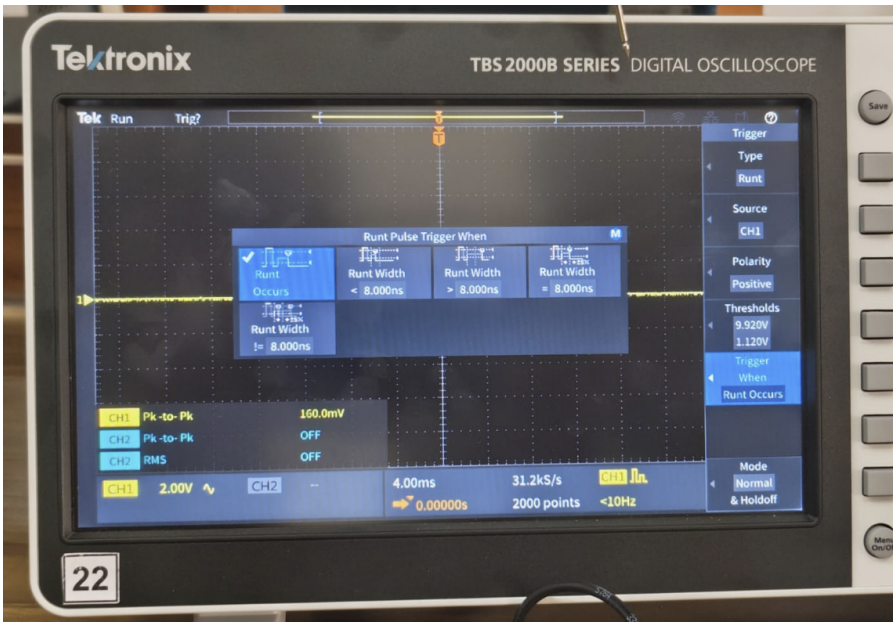
We can adjust parameters like the source of the signal, the type of edge (falling or rising edge), and the level (height of the edge).

The different pulse width trigger options are shown in the figure below.



As we can see, we can adjust parameters like the source of the signal, the polarity of the pulse to be captured, the height of the pulse and the width of the pulse to be captured.

The different runt trigger options are shown in the figure below.



As we can see, we can adjust parameters to define what kind of runt to capture such as width, polarity and threshold.